



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI-Assisted C Programming for Arduino

TGS Ref No: TGS-2023039924

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	Original source release	Original 160-slide trainer reference	Tertiary Infotech Academy
2.0	13 September 2026	Rebuilt C foundations, UNO Q safety, 14 independent activities, explicit contracts/fault tests and source-derived slide references	Dr Alfred Ang

TABLE OF CONTENTS

How to Use This Guide.....	6
Course Outcomes and Assessment.....	6
Topic 1 — Specification, C foundations and safe hardware /Slides 17-62.....	6
Specification becomes a component contract /Slides 18-20.....	6
UNO Q assigns work to two processors /Slides 21-23.....	7
Keep analogue inputs inside their range /Slides 24-26.....	7
C logic lives behind a C++ sketch /Slides 27-29.....	8
Variables encode value and unit /Slides 30-32.....	8
Integer arithmetic needs promotion /Slides 33-35.....	8
A function makes state visible /Slides 36-38.....	9
Lifetime is not the same as scope /Slides 39-41.....	9
Compiler output is executable evidence /Slides 42-44.....	10
Dynamic allocation has an explicit failure path /Slides 45-47.....	10
Activity 1 — Specify and model a doorbell /Slides 48-52.....	11
Step-by-step Activity 1.....	11
Vibe-coding prompts /same-source prompt PDF.....	12
Reference C logic and host assertions.....	12
Complete Arduino C++ adapter /physical extension.....	12
Activity 2 — Validate a GPIO truth table /Slides 53-57.....	13
Step-by-step Activity 2.....	13
Vibe-coding prompts /same-source prompt PDF.....	14
Reference C logic and host assertions.....	14
Complete Arduino C++ adapter /physical extension.....	14
Activity 3 — Schedule events without delay /Slides 58-62.....	15
Step-by-step Activity 3.....	15
Vibe-coding prompts /same-source prompt PDF.....	16
Reference C logic and host assertions.....	16
Complete Arduino C++ adapter /physical extension.....	16
Topic 2 — Modular development and AI-assisted review /Slides 63-114.....	17
Header guards share one declaration /Slides 64-66.....	17
AI review is bounded by evidence /Slides 67-69.....	18
Separate acquisition from conversion /Slides 70-72.....	18
Macros can evaluate twice /Slides 73-75.....	18
Preprocessor choices change the build /Slides 76-78.....	19
Warnings expose narrowing mistakes /Slides 79-81.....	19
Documentation comments state contracts /Slides 82-84.....	20
Host files are not MCU storage APIs /Slides 85-87.....	20
Version evidence identifies the build /Slides 88-90.....	21
C++ references wrap caller-owned state /Slides 91-93.....	21
RAII ties a resource to object lifetime /Slides 94-96.....	22
C and C++ compile different source contracts /Slides 97-99.....	22
Activity 4 — Scale an ADC reading to PWM /Slides 100-104.....	23
Step-by-step Activity 4.....	23
Vibe-coding prompts /same-source prompt PDF.....	24
Reference C logic and host assertions.....	24

Complete Arduino C++ adapter /physical extension.....	24
Activity 5 — Review modular sensor conversion /Slides 105-109.....	25
Step-by-step Activity 5.....	25
Vibe-coding prompts /same-source prompt PDF.....	26
Reference C logic and host assertions.....	26
Complete Arduino C++ adapter /physical extension.....	26
Activity 6 — Repair a header and build contract /Slides 110-114.....	27
Step-by-step Activity 6.....	27
Vibe-coding prompts /same-source prompt PDF.....	28
Reference C logic and host assertions.....	28
Complete Arduino C++ adapter /physical extension.....	28
Topic 3 — Controls, memory and data structures /Slides 116-158.....	29
If-else encodes a boundary policy /Slides 117-119.....	29
Switch-case selects operating modes /Slides 120-122.....	29
For loops walk bounded arrays /Slides 123-125.....	30
While and do-while differ at entry /Slides 126-128.....	30
Pointers require ownership and length /Slides 129-131.....	31
Ring indexing keeps memory fixed /Slides 132-134.....	31
Strings reserve a terminator byte /Slides 135-137.....	32
Structs package values, not wire bytes /Slides 138-140.....	32
Unions and bitfields need a tag /Slides 141-143.....	33
Activity 7 — Implement fan modes and hysteresis /Slides 144-148.....	33
Step-by-step Activity 7.....	33
Vibe-coding prompts /same-source prompt PDF.....	34
Reference C logic and host assertions.....	34
Complete Arduino C++ adapter /physical extension.....	34
Activity 8 — Use an array-backed ring buffer /Slides 149-153.....	35
Step-by-step Activity 8.....	35
Vibe-coding prompts /same-source prompt PDF.....	36
Reference C logic and host assertions.....	36
Complete Arduino C++ adapter /physical extension.....	37
Activity 9 — Encode bounded status records /Slides 154-158.....	37
Step-by-step Activity 9.....	37
Vibe-coding prompts /same-source prompt PDF.....	38
Reference C logic and host assertions.....	38
Complete Arduino C++ adapter /physical extension.....	39
Topic 4 — Contracts, interoperability and fault evidence /Slides 159-201.....	39
Serial framing precedes parsing /Slides 160-162.....	39
An exact parser rejects extra tokens /Slides 163-165.....	40
Freshness is part of interoperability /Slides 166-168.....	40
I2C contracts include electrical levels /Slides 169-171.....	40
SD logging needs write and readback /Slides 172-174.....	41
Fault gates must fail deterministically /Slides 175-177.....	41
A test oracle must be independent /Slides 178-180.....	42
Functionality indicators are observable /Slides 181-183.....	42
Bridge RPC is a value boundary /Slides 184-186.....	43

Activity 10 — Parse a bounded serial command /Slides 187-191.....	43
Step-by-step Activity 10.....	44
Vibe-coding prompts /same-source prompt PDF.....	44
Reference C logic and host assertions.....	45
Complete Arduino C++ adapter /physical extension.....	45
Activity 11 — Integrate sensor display and logging /Slides 192-196.....	46
Step-by-step Activity 11.....	46
Vibe-coding prompts /same-source prompt PDF.....	47
Reference C logic and host assertions.....	47
Complete Arduino C++ adapter /physical extension.....	47
Activity 12 — Build a fault-injection gate /Slides 197-201.....	48
Step-by-step Activity 12.....	48
Vibe-coding prompts /same-source prompt PDF.....	49
Reference C logic and host assertions.....	49
Complete Arduino C++ adapter /physical extension.....	49
Topic 5 — User-aligned documentation and bounded extension /Slides 203-231.....	50
Trace requirements into maintenance docs /Slides 204-206.....	50
Architecture diagram is not a wiring plan /Slides 207-209.....	50
Local AI proposes; deterministic C decides /Slides 210-212.....	51
User documentation needs honest limits /Slides 213-215.....	51
Document changes preserve the contract /Slides 216-218.....	52
Doxygen tags document the public interface /Slides 219-221.....	52
Activity 13 — Document an irrigation interface /Slides 222-226.....	53
Step-by-step Activity 13.....	53
Vibe-coding prompts /same-source prompt PDF.....	54
Reference C logic and host assertions.....	54
Complete Arduino C++ adapter /physical extension.....	54
Activity 14 — Bound a UNO Q local-AI suggestion /Slides 227-231.....	55
Step-by-step Activity 14.....	55
Vibe-coding prompts /same-source prompt PDF.....	56
Reference C logic and host assertions.....	56
Complete Arduino C++ adapter /physical extension.....	56
Optional UNO Q Bridge client /bounded LED lease.....	57
Quick Command Reference.....	58
Support and Assessment Flow.....	58
Sources and Verification Limits.....	58

How to Use This Guide

Version 2.0 / TGS-2023039924. Slides teach architecture, runtime, exact contracts, failure and verification. This guide and each independent Activity README contain the detailed procedures. The core logic is C-compatible; Arduino .ino files are C++ runtime wrappers, not pure C programs.

Host prerequisites: C compiler (cc/clang/gcc), terminal and editor. Physical extension: exact UNO Q variant, selected Arduino board core, USB cable and reviewed 3.3 V-safe peripherals. AI review can be demonstrated or performed manually; no paid API or real secret is required.

Safety: A0-A5 are not 5 V-tolerant analogue inputs; remain within 0..VREF+ approximately 3.3 V. D3/PB0 maximum 3.6 V; A4/A5 I2C pull-ups 3.3 V only. Any real fan, buzzer, valve or pump requires an approved driver and relevant protection. Use LED surrogates for initial practice.

Course Outcomes and Assessment

LO1: Determine basic software components according to functional and business specifications (K1/A1).

LO2: Apply software design methodologies and tools in accordance with organisational practices (K2/A2).

LO3: Select controls, elements and features to meet design objectives (K3/A3).

LO4: Examine functionality and interoperability of software components (K4/A4).

LO5: Generate detailed design documentation mapped to user specifications (K5/A5).

Two days / 16 hours: 14 training and 2 assessment hours. Day 2 WA 4:00-5:00 PM, PP 5:00-6:00 PM; each original paper has five questions/tasks. Open-book and individual. Follow the current assessor-issued paper and submission route; trainer answer keys are not learner material.

Topic 1 — Specification, C foundations and safe hardware / Slides 17-62

K1/A1 / LO1: Determine basic software components according to functional and business specifications (K1/A1).

Specification becomes a component contract / Slides 18-20

Input: Separate electrical raw level from pressed intent.

State: Two persistent variables implement a timed interval.

Function: doorbell() has explicit inputs and return value.

Evidence: 1999/2000 ms tests distinguish the deadline.

Contract field	Exact rule
Requirement	Visitor gets visible and audible feedback.
Data type	uint32_t timestamps; int Boolean request.
Function boundary	Pure logic does not configure pins.
Business limit	Buzzer driver rating is a hardware prerequisite.

R1 button press -> request outputs

R2 LED and buzzer share 2000 ms

R3 loop remains responsive

input: pressed, now_ms

state: active, started_ms

output: Boolean request

Failure: Ambiguous duration. Response: Define whether repeated presses restart the interval before coding.

Acceptance: A trace matrix maps R1/R2/R3 to function, test and physical output evidence.

UNO Q assigns work to two processors /Slides 21-23

Linux: QRB2210 quad Cortex-A53 up to 2 GHz runs Debian Linux.

MCU: STM32U585 Cortex-M33 up to 160 MHz handles deterministic I/O.

Boundary: Bridge RPC carries explicit values, not shared raw pointers.

Scope: Core activities test C logic; local AI is an optional extension.

Contract field	Exact rule
ABX00162	2 GB RAM / 16 GB storage variant.
ABX00173	4 GB RAM / 32 GB storage variant.
MCU memory	Arduino manual states 2 MB flash / 786 kB SRAM.
Authority	Current ABX00162 manual, updated 17 June 2026.

Debian Linux / QRB2210

Python app, optional local model

Bridge RPC

STM32U585 / Arduino sketch

bounded control logic

3.3 V-safe MCU I/O

Failure: UNO R3 assumption. Response: Do not import ATmega328P, 5 V analogue or AVR memory assumptions.

Acceptance: Identify exact board variant and core/library versions in the verification record.

Keep analogue inputs inside their range /Slides 24-26

Analogue: A0-A5 are not 5 V-tolerant analogue inputs.

D3: The special PB0 limit forbids a blanket digital 5 V rule.

Pull-ups: External I2C pull-ups must match the approved 3.3 V interface.

Power: USB PD requests 5 V/3 A; VIN range is 7-24 V.

Contract field	Exact rule
Signal review	Verify sensor output level before connecting.
Actuation	Use rated driver; never power a motor from a GPIO.
Qwiic	I2C4 uses PD13 SDA / PD12 SCL at 3.3 V.
Manual evidence	Consult board pinout for exact selected pin.

A0..A5 -> STM32 analogue inputs

valid voltage: GND .. VREF+

VREF+ approximately 3.3 V

D3/PB0: maximum 3.6 V

A4/A5 I2C pullups: 3.3 V only

Failure: Over-voltage risk. Response: Stop wiring; reduce or translate voltage with an approved circuit.

Acceptance: Trainer signs a pin/voltage/driver review before any physical upload.

C logic lives behind a C++ sketch /Slides 27-29

Language: An .ino file is compiled as Arduino C++, not pure C.

C module: logic.c remains C-compatible and host-testable.

Header: extern "C" guards preserve linkage when called from C++.

Adapter: setup/loop and Arduino APIs stay in the sketch.

Contract field	Exact rule
logic.c	C11 host compilation; no Arduino includes.
logic.h	Shared declarations and fixed-width types.
sketch.ino	C++ runtime wrapper around pure logic.
Host test	Does not prove board upload or pin behaviour.

```
#include "logic.h"
void setup() { /* configure I/O */ }
void loop() {
  int request = alarm(25, 30);
  /* adapter writes request */
}
```

Failure: Linker name mismatch. Response: Check header linkage guards and compile .c as C.

Acceptance: C tests build with cc; selected-core sketch compile succeeds independently.

Variables encode value and unit /Slides 30-32

Unit: 250 means 25.0 degrees; encode the unit in the name.

Range: int16_t supports negatives and the chosen sensor range.

Time: Unsigned milliseconds support modular elapsed-time checks.

Constant: const describes intent, not necessarily a flash location.

Contract field	Exact rule
Temperature	Allowed -400..1250 tenths in integration activity.
Timestamp	uint32_t wraps modulo 2 ³² .
Request	0/1 only; do not use a temperature as Boolean.
Platform	sizeof(int) is target-dependent; inspect instead of assuming.

```
#include <stdint.h>
int16_t temp_tenths = 250;
uint32_t sample_ms = 1000u;
uint8_t fan_request = 0u;
const int16_t on_tenths = 300;
```

Failure: Degrees versus counts. Response: Convert at the acquisition boundary; reject impossible ranges.

Acceptance: 250 is displayed as 25.0 degrees only after explicit unit formatting.

Integer arithmetic needs promotion /Slides 33-35

Promotion: Perform multiplication in uint32_t before narrowing.

Rounding: Adding half the denominator rounds to nearest.

Clamping: Negative and over-range raw values are bounded first.

Configuration: 4095 is an explicit teaching setting, not an assumed default.

Contract field	Exact rule
adc_max	1..65535; zero is invalid.
raw	Signed input clamped to 0..adc_max.
Result	uint8_t output 0..255.
Physical	PWM pin and frequency depend on selected core/pin.

```
raw = 2048; adc_max = 4095;
numerator = raw * 255 + adc_max/2;
PWM = numerator / adc_max;
PWM = 128;
maximum numerator < 2^32
```

Failure: Narrow overflow. Response: Cast the operand before multiplication, not the overflowing result.

Acceptance: 0, midpoint, full scale and invalid denominator return 0,128,255,0.

A function makes state visible /Slides 36-38

Parameter: limit is passed, not read from a hidden global.

Return: Equality is defined as alarm-on.

Purity: No pin writes or logging inside the function.

Testability: The same contract runs on a host and MCU.

Contract field	Exact rule
Inputs	Signed whole degrees; both values share a unit.
Output	Exactly 0 or 1.
Side effects	None.
Boundary	29 below; 30 at; 31 above a limit of 30.

```
int alarm(int temperature, int limit) {
    return temperature >= limit;
}
/* caller chooses the threshold */
int request = alarm(30, 30);
```

Failure: Wrong inequality. Response: Check equality deliberately instead of testing only far-away values.

Acceptance: alarm(29,30)=0 and alarm(30,30)=1 are separate assertions.

Lifetime is not the same as scope /Slides 39-41

Static: last_ms persists across calls and is zero-initialised.

Automatic: local_request exists only during the function call.

Scope: Both names are private to their declaration context.

Ownership: Persistent state must have a documented reset operation.

Contract field	Exact rule
Automatic storage	Never return the address of a local variable.

Contract field	Exact rule
Static state	Survives calls; can complicate test isolation.
Explicit struct	Preferred when multiple instances are needed.
Initialisation	Tests establish a known initial state.

```
static uint32_t last_ms;
void poll(uint32_t now) {
    int local_request = 0;
    if (due(now, &last_ms, 100u))
        local_request = 1;
    /* local_request dies on return */
}
```

Failure: Dangling pointer. Response: Return a value or use caller-owned storage instead of a local address.

Acceptance: Two calls retain last_ms but each call starts local_request at zero.

Compiler output is executable evidence /Slides 42-44

Preprocess: Headers and macros become a translation unit.

Compile: Types and syntax are checked per unit.

Link: External references must resolve across units.

Run: A compiled binary can still implement wrong requirements.

Contract field	Exact rule
Host gate	-std=c11 -Wall -Wextra -Werror -pedantic.
Test gate	Nonzero exit means failed acceptance.
Board gate	Record board/core/library selection.
Evidence limit	Host compile is not measured hardware upload.

```
source .c -> preprocessing
translation -> object .o
linking -> host executable
warnings -> review gate
assertions -> run-time evidence
board compile -> separate target gate
```

Failure: Compile-only confidence. Response: Run boundary assertions and board-specific checks, not just syntax.

Acceptance: Every activity prints its PASS label only after all assertions execute.

Dynamic allocation has an explicit failure path /Slides 45-47

Allocation: Heap allocation can fail even after compilation succeeds.

Bounds: Validate count and product size before allocation.

Ownership: One owner releases the allocation exactly once.

Embedded: Fixed-capacity arrays avoid allocation in the control loop.

Contract field	Exact rule
Header	<stdlib.h> declares malloc/free.
Size	Check count<=SIZE_MAX/sizeof *values before multiplying.

Contract field	Exact rule
Failure	NULL is not a valid array to index.
realloc	Use a temporary pointer; preserve original on failure.

```
int *values = malloc(count * sizeof *values);
if (values == NULL) { /* safe failure */ }
/* use only within validated count */
free(values);
values = NULL;
/* core activities prefer fixed arrays */
```

Failure: Leak or double free. Response: Document one owner, one release and a NULL-safe failure path.

Acceptance: Core ring-buffer tests require no heap allocation; optional host allocation tests cover failure handling.

Activity 1 — Specify and model a doorbell /Slides 48-52

Goal: A visitor presses a button. The LED and a low-power buzzer request remain active for 2 seconds without freezing the loop. K1/A1; suggested practice 25 minutes. Independent folder: labs/activity-01-requirements-doorbell.

Exact contract: Input pressed is 0/1; timestamp is uint32_t milliseconds. A new press restarts the 2000 ms interval; output is 0 at the deadline.

Step-by-step Activity 1

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Input pressed is 0/1; timestamp is uint32_t milliseconds. A new press restarts the 2000 ms interval; output is 0 at the deadline.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Use a 3.3 V-safe button input and LED resistor. Use a driver for a buzzer if its current exceeds the pin capability. Capture both output changes at 0, 1999 and 2000 ms; do not infer timing from host tests.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Input pressed is 0/1; timestamp is uint32_t milliseconds. A new press restarts the 2000 ms interval; output is 0 at the deadline. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Specify and model a doorbell. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-01-requirements-doorbell/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int doorbell(uint32_t now, int pressed, uint32_t *started, int *active) {
    if (pressed) { *started = now; *active = 1; }
    if (*active && (uint32_t)(now - *started) >= 2000u) *active = 0;
    return *active;
}
uint32_t start=0; int active=0;
assert(doorbell(10,1,&start,&active)==1);
assert(doorbell(2009,0,&start,&active)==1);
assert(doorbell(2010,0,&start,&active)==0);
assert(doorbell(UINT32_MAX-100,1,&start,&active)==1);
assert(doorbell(1899,0,&start,&active)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
static uint32_t started=0,last=0;
static int active=0,previous=0;
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    Bridge.begin(); Monitor.begin(115200);
    // For ADC activities, verify analogReadResolution(12) is supported,
    // enable it here and keep ADC_MAX consistent before physical use.
}
void loop() {
    int request=doorbell((uint32_t)millis(), digitalRead(BUTTON_PIN)==LOW, &started, &active);
    digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
```

```

Monitor.println(request);
// Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Five assertions pass; activation is 1 before the deadline and 0 at it.

Physical-board check: Use a 3.3 V-safe button input and LED resistor. Use a driver for a buzzer if its current exceeds the pin capability. Capture both output changes at 0, 1999 and 2000 ms; do not infer timing from host tests.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 2 — Validate a GPIO truth table /Slides 53-57

Goal: An active-low push button and an enable switch control a status indicator. Separate electrical level from logical meaning. K1/A1; suggested practice 20 minutes. Independent folder: labs/activity-02-gpio-truth-table.

Exact contract: raw_high 0 means pressed; enabled 0 always suppresses the output. Invalid levels return 0.

Step-by-step Activity 2

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: raw_high 0 means pressed; enabled 0 always suppresses the output. Invalid levels return 0.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Connect button only to the approved 3.3 V MCU input and ground, using INPUT_PULLUP. Read both released and pressed levels in Serial Monitor; verify LED polarity separately.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: raw_high 0 means pressed; enabled 0 always suppresses the output. Invalid levels return 0. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Validate a GPIO truth table. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-02-gpio-truth-table/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int indicator(int raw_high, int enabled) {
    if ((raw_high != 0 && raw_high != 1) ||
        (enabled != 0 && enabled != 1)) return 0;
    return !raw_high && enabled;
}
assert(indicator(0,1)==1); assert(indicator(1,1)==0);
assert(indicator(0,0)==0); assert(indicator(1,0)==0);
assert(indicator(2,1)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
const int ENABLE_PIN=4; // Switch to GND enables; reviewed3.3V input.
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    pinMode(ENABLE_PIN,INPUT_PULLUP);
    Bridge.begin(); Monitor.begin(115200);
    // For ADC activities, verify analogReadResolution(12) is supported,
    // enable it here and keep ADC_MAX consistent before physical use.
}
void loop() {
```

```
int request=indicator(digitalRead(BUTTON_PIN)==HIGH, digitalRead(ENABLE_PIN)==LOW);
digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
Monitor.println(request);
// Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: All four truth-table combinations and one invalid value pass.

Physical-board check: Connect button only to the approved 3.3 V MCU input and ground, using INPUT_PULLUP. Read both released and pressed levels in Serial Monitor; verify LED polarity separately.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 3 — Schedule events without delay /Slides 58-62

Goal: A status LED tick must not block sensor polling. Decide whether one periodic task is due from elapsed time. K1/A1 K3/A3; suggested practice 25 minutes. Independent folder: labs/activity-03-event-scheduler.

Exact contract: period must be nonzero and below 2^{31} ms; next execution follows the actual observed timestamp (no backlog replay).

Step-by-step Activity 3

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: period must be nonzero and below 2^{31} ms; next execution follows the actual observed timestamp (no backlog replay).
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Upload the approved LED-only wrapper; print sensor polling and tick timestamps. Measure that the main loop continues between ticks. Hardware latency is a separate record from the host due() test.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: period must be nonzero and below 2^{31} ms; next execution follows the actual observed timestamp (no backlog replay). Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Schedule events without delay. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-03-event-scheduler/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int due(uint32_t now, uint32_t *last, uint32_t period) {
    if (period == 0u || period >= 0x80000000u) return 0;
    if ((uint32_t)(now - *last) < period) return 0;
    *last = now; return 1;
}
uint32_t last=0;
assert(due(99,&last,100)==0); assert(due(100,&last,100)==1);
assert(last==100); assert(due(101,&last,100)==0);
assert(due(500,&last,0)==0);
last=UINT32_MAX-50; assert(due(49,&last,100)==1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
static uint32_t started=0,last=0;
static int active=0,previous=0;
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    Bridge.begin(); Monitor.begin(115200);
    // For ADC activities, verify analogReadResolution(12) is supported,
    // enable it here and keep ADC_MAX consistent before physical use.
```

```

}
void loop() {
  int request=due((uint32_t)millis(), &last, 100u);
  digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
  Monitor.println(request);
  // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Six checks pass including rollover and invalid period.

Physical-board check: Upload the approved LED-only wrapper; print sensor polling and tick timestamps. Measure that the main loop continues between ticks. Hardware latency is a separate record from the host due() test.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Topic 2 — Modular development and AI-assisted review /Slides 63-114

K2/A2 /LO2: Apply software design methodologies and tools in accordance with organisational practices (K2/A2).

Header guards share one declaration /Slides 64-66

Guard: Prevents duplicate declarations of types in one translation unit.

Declaration: Header specifies the callable interface.

Implementation: Function body belongs in logic.c.

Linkage: A C++ caller sees C linkage only inside the guard.

Contract field	Exact rule
Include	Both implementation and caller include the same header.
Types	Use matching types in declaration and definition.
Constants	Avoid multiple mutable global definitions in a header.
Build proof	Include the header twice in the host test.

```

#ifndef SENSOR_LOGIC_H
#define SENSOR_LOGIC_H
#ifdef __cplusplus
extern "C" {
#endif
int alarm(int temperature,int limit);
/* close extern C and guard */

```

Failure: Conflicting prototypes. Response: Remove copied declarations; rebuild all dependent translation units.

Acceptance: Duplicate inclusion compiles; mismatched parameter types fail the warning gate.

AI review is bounded by evidence /Slides 67-69

Context: Supply exact contracts and target constraints, not just a task name.

Review: Require explanations tied to lines and requirements.

Privacy: Use mock sensor data; omit credentials and client identities.

Authority: The assistant proposes; maintainer accepts only verified changes.

Contract field	Exact rule
Permitted	Explain, refactor and propose tests.
Forbidden	Silently change thresholds, units or hardware pins.
Evidence	Save prompt, proposal, decision and test outcome.
Failure mode	Plausible generated code can still be unsafe.

Input: specification + logic.h

Ask: find overflow and edge cases

Output: patch + stated assumptions

Human: compare interface and units

Compiler: strict warnings

Tests: boundaries and invalid inputs

Failure: Invented API. Response: Open installed official library documentation and compile a minimal call.

Acceptance: A proposed patch preserves all prior tests and adds one new boundary assertion.

Separate acquisition from conversion /Slides 70-72

Acquisition: Arduino API reads a hardware-dependent count.

Conversion: C logic uses an explicit maximum.

Presentation: Serial output is outside the pure function.

Calibration: A percentage of ADC range is not lux.

Contract field	Exact rule
Raw value	Count in a configured resolution range.
maximum	Explicit denominator, 1..65535.
Return	-1 for invalid maximum; otherwise 0..100.
Documentation	Record units at each boundary.

analogRead(A0) -> raw count

light_percent(raw, maximum)

validated percentage -> Serial

host mock -> same conversion

MCU wrapper -> same C logic

Failure: Sensor claim without calibration. Response: Label output as scaled ADC percentage, not physical light intensity.

Acceptance: Mock 2048/4095 rounds to 50%; real calibration remains a separate experiment.

Macros can evaluate twice /Slides 73-75

Expansion: A macro substitutes text before compilation.

Side effect: i++ can appear in condition and selected branch.

Inline: Function arguments are evaluated once each.

Parentheses: Protect precedence but do not fix repeated evaluation.

Contract field	Exact rule
Macro argument	Never pass an increment to a repeated-argument macro.
Constant macro	Suitable for a compile-time numeric setting.
Typed function	Enforces parameter and return types.
Review	Inspect preprocessed meaning, not only the macro name.

```
#define BAD_MAX(a,b) ((a)>(b)?(a):(b))
/* BAD_MAX(i++, 10) is unsafe */
static inline int max_int(int a,int b) {
    return a>b ? a:b;
}
```

Failure: Surprising increment. Response: Replace function-like macro with a typed helper.

Acceptance: Call max_int with precomputed values and assert one deterministic result.

Preprocessor choices change the build /Slides 76-78

Configuration: A build flag can override the fallback constant.

Validation: Reject invalid configuration before runtime.

Conditional: #if selects compiled text, not a runtime branch.

Traceability: Record flags in the build evidence.

Contract field	Exact rule
Default	100 milliseconds is the teaching period.
Override	Compiler -D is a build configuration.
Invalid	Zero must fail preprocessing.
Portability	No board identity is inferred from a macro alone.

```
#ifndef SAMPLE_PERIOD_MS
#define SAMPLE_PERIOD_MS 100u
#endif
#if SAMPLE_PERIOD_MS == 0
#error Sample period must be nonzero
#endif
```

Failure: Different binaries. Response: Compare exact build flags when two learners observe different periods.

Acceptance: Zero configuration is a negative build test; default configuration passes.

Warnings expose narrowing mistakes /Slides 79-81

Intermediate: scaled needs more than 16 bits at full raw scale.

Conversion: A cast can hide warnings without proving correctness.

Validation: Narrow only after proving range 0..255.

Target: Integer sizes vary; fixed-width intermediates express intent.

Contract field	Exact rule
Maximum product	65535*255 = 16711425.
Result range	0..255 after division.
Warning gate	Enable strict baseline and review extra conversion warnings.
Assertion	Test maximum input, not only small values.

```
uint32_t raw = 65535u;
uint32_t scaled = raw * 255u;
uint8_t request = (uint8_t)255u;
/* not: uint16_t scaled = raw*255 */
/* validate before explicit narrowing */
```

Failure: Cast-as-fix. Response: Prove the intermediate bound before adding an explicit cast.

Acceptance: Maximum range computation fits uint32_t and returns 255 after scaling.

Documentation comments state contracts /Slides 82-84

User need: Dry soil can request watering; faults must suppress it.

Unit: Percentage and milliseconds are explicit.

Return: 0 is safe-off, including invalid data.

Limit: A request function is not a complete pump safety controller.

Contract field	Exact rule
Header comment	Defines public preconditions and output.
Implementation comment	Explains non-obvious invariants, not syntax.
Report	States limits such as missing maximum runtime.
Change	Update tests and docs when threshold changes.

```
/* percent: 0..100; age: milliseconds
 * returns 1 only when dry and fresh
 * invalid inputs return 0
 * no GPIO side effects
 */
int water_request(int percent,unsigned age,int ok);
```

Failure: Comment/code divergence. Response: Trace every threshold statement to the implementation and assertion.

Acceptance: Documentation says <30 and the 29/30 tests prove that exact inequality.

Host files are not MCU storage APIs /Slides 85-87

Host: stdio file I/O reads local mock CSV in a desktop program.

Bound: fgets limits bytes; a truncated line still needs rejection.

Error: Check open/read/close outcomes as required.

MCU: SD logging uses a board/library-specific adapter, not assumed fopen.

Contract field	Exact rule
Input	Local mock data; no real personal records.
EOF	End-of-file differs from malformed line.
Parser	Check conversion count and permitted range.

Contract field	Exact rule
Target	Physical SD write/readback is separate evidence.

```
void read_mock(void) {
FILE *fp = fopen("samples.csv", "r");
if (fp == NULL) return; /* stop: no valid handle */
char line[64];
while (fgets(line, sizeof line, fp)) {
/* validate complete bounded record */
}
fclose(fp);
}
```

Failure: Partial record. Response: Detect missing newline/fields and reject rather than silently parse a prefix.

Acceptance: Valid mock row parses; malformed and overlong rows fail without a GPIO side effect.

Version evidence identifies the build /Slides 88-90

Artifact: A version label identifies course content, not target firmware SHA.

Source: Record the reviewed revision when available.

Environment: Core/library differences can change supported APIs.

Honesty: NOT RUN is evidence of a limit, not a failure to conceal.

Contract field	Exact rule
Host record	Compiler version plus test command/output.
MCU record	Board/core and sketch compile result.
Physical record	Wiring/photo/readback/timestamps.
Reproduction	Another learner can identify the same configuration.

```
course version: 2.0
source revision: recorded at release
board model: exact variant
core and library versions: recorded
host compiler: recorded
hardware result: NOT RUN until measured
```

Failure: Unreproducible success. Response: Add exact environment and input data to the verification record.

Acceptance: A release record distinguishes generated tests, executed tests and physical measurements.

C++ references wrap caller-owned state /Slides 91-93

Reference: A reference names an existing object; it is not an owning copy.

Lifetime: Caller-owned Sample must outlive SensorView.

Const: View cannot modify the referenced sample.

Scope: This lightweight C++ adapter does not replace the C core.

Contract field	Exact rule
C core	Sample and functions remain C-compatible.

Contract field	Exact rule
C++ wrapper	Classes/references belong in .cpp or .ino, not logic.c.
Ownership	No allocation is required for this view.
Getter	Returns a value and documents temperature unit.

```
class SensorView {
    const Sample& sample;
public:
    explicit SensorView(const Sample& s):sample(s) {}
    int temperature() const { return sample.temp_tenths; }
};
```

Failure: Dangling reference. Response: Do not bind a persistent view to a local object that is about to die.

Acceptance: Compile wrapper as C++; keep host C modules compiled as C11.

RAII ties a resource to object lifetime /Slides 94-96

Acquire: Constructor tries to open a host file.

Release: Destructor closes an acquired resource.

Copy: Disabled copying prevents two owners closing one handle.

Boundary: Host example teaches RAII, not an assumed MCU SD API.

Contract field	Exact rule
Language	RAII/class/destructor are C++ concepts.
Failure	Constructor must expose/check unsuccessful open.
Ownership	One object owns the handle.
Target	Use actual selected SD library in board adapters.

```
class HostFile {
    FILE* fp;
public:
    explicit HostFile(const char* path):fp(fopen(path,"r")) {}
    ~HostFile() { if(fp) fclose(fp); }
    HostFile(const HostFile&) = delete;
    HostFile& operator=(const HostFile&) = delete;
};
```

Failure: Double close. Response: Use one owner and disable unsafe copying; document lifetime.

Acceptance: A C++ host wrapper compiles; absence of a file is handled without a leaked handle.

C and C++ compile different source contracts /Slides 97-99

C: Structures, pointers and functions form the hardware-independent core.

C++: References, const member functions and RAII fit thin adapters.

Linkage: extern C avoids a C++ symbol-name mismatch.

Education: Google C++ material and requested tutorials supplement, not replace assessed C.

Contract field	Exact rule
Google primary	developers.google.com/edu/c++/getting-started .
Requested context	learn-cpp.org and learncpp.com learning references.
Core scope	All five approved software-design outcomes retained.
Evidence	Compile C and C++ units with the correct tools.

```
logic.c: cc -std=c11
adapter.cpp: c++ -std=c++17
logic.h: guarded extern "C"
.ino: Arduino-generated C++ translation
no classes/references/destructors in C module
```

Failure: Compiling C as C++. Response: Do not conceal unsupported C constructs by changing source extensions.

Acceptance: The same guarded header links a separately compiled C object into a C++ caller.

Activity 4 — Scale an ADC reading to PWM /Slides 100-104

Goal: A greenhouse light monitor maps a configurable ADC range to a bounded PWM command without assuming UNO R3 ADC resolution. K2/A2 K3/A3; suggested practice 25 minutes.

Independent folder: labs/activity-04-adc-pwm.

Exact contract: raw is clamped to 0..adc_max; adc_max 1..65535; result rounded to nearest 0..255. Return 0 for invalid adc_max.

Step-by-step Activity 4

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: raw is clamped to 0..adc_max; adc_max 1..65535; result rounded to nearest 0..255. Return 0 for invalid adc_max.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Use analogReadResolution only after checking the installed core supports the chosen setting; document adc_max from that setting. A0-A5 input must remain between ground and approximately 3.3 V VREF+. Use a low-power LED/resistor on a verified PWM pin; no motor directly.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: raw is clamped to 0..adc_max; adc_max 1..65535; result rounded to nearest 0..255. Return 0 for invalid adc_max. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Scale an ADC reading to PWM. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-04-adc-pwm/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
uint8_t pwm_scale(int32_t raw, uint32_t adc_max) {
    if (adc_max == 0 || adc_max > 65535u) return 0;
    if (raw < 0) raw = 0;
    if ((uint32_t)raw > adc_max) raw = (int32_t)adc_max;
    return (uint8_t)((uint32_t)raw * 255u + adc_max/2u) / adc_max;
}
assert(pwm_scale(0,4095)==0); assert(pwm_scale(4095,4095)==255);
assert(pwm_scale(2048,4095)==128); assert(pwm_scale(-1,4095)==0);
assert(pwm_scale(5000,4095)==255); assert(pwm_scale(1,0)==0);
assert(pwm_scale(65535,65535)==255);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
const int PWM_PIN=9; // D9/PB8 TIMER4 CH3; external3.3V LED +resistor.
const uint32_t ADC_MAX=4095u;
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    pinMode(PWM_PIN,OUTPUT); analogWrite(PWM_PIN,0);
    Bridge.begin(); Monitor.begin(115200);
}
```

```

analogWriteResolution(8);
analogReadResolution(12); // Selected UNO Q core must support this setting.
}
void loop() {
  int raw=analogRead(A0);
  if(raw<0) { analogWrite(PWM_PIN,0); digitalWrite(LED_BUILTIN,LOW);
  Monitor.println("ADC_ERROR"); return; }
  int request=pwm_scale(raw, ADC_MAX);
  analogWrite(PWM_PIN,request);
  digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
  Monitor.print("raw="); Monitor.print(raw);
  Monitor.print(", adc_max="); Monitor.print(ADC_MAX);
  Monitor.print(", scaled="); Monitor.println(request);
  // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Seven assertions pass; midpoint 2048/4095 maps to 128.

Physical-board check: Use analogReadResolution only after checking the installed core supports the chosen setting; document adc_max from that setting. A0-A5 input must remain between ground and approximately 3.3 V VREF+. Use a low-power LED/resistor on a verified PWM pin; no motor directly.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 5 — Review modular sensor conversion /Slides 105-109

Goal: An AI assistant proposes a light percentage conversion. Keep raw acquisition in the Arduino wrapper and pure conversion in a C module. K2/A2; suggested practice 25 minutes. Independent folder: labs/activity-05-modular-sensor.

Exact contract: Maximum 1..65535; clamped raw input; rounded integer percentage 0..100. No Serial, GPIO or allocation in this module.

Step-by-step Activity 5

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Maximum 1..65535; clamped raw input; rounded integer percentage 0..100. No Serial, GPIO or allocation in this module.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.

6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.

7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Build the MCU adapter with the selected board core, then use a 3.3 V-safe LDR divider on A0. Record raw values under two lighting conditions and explain why percentage is not calibrated lux.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Maximum 1..65535; clamped raw input; rounded integer percentage 0..100. No Serial, GPIO or allocation in this module. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Review modular sensor conversion. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-05-modular-sensor/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int light_percent(int raw, unsigned maximum) {
    if (maximum==0 || maximum>65535u) return -1;
    if (raw<0) raw=0;
    if ((unsigned)raw>maximum) raw=(int)maximum;
    return (int)((unsigned)raw*100u+maximum/2u)/maximum;
}
assert(light_percent(0,4095)==0); assert(light_percent(4095,4095)==100);
assert(light_percent(2048,4095)==50); assert(light_percent(-4,4095)==0);
assert(light_percent(10,0)==-1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
```

```

const int BUTTON_PIN=2;
const uint32_t ADC_MAX=4095u;
void setup() {
  pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
  Bridge.begin(); Monitor.begin(115200);
  analogReadResolution(12); // Selected UNO Q core must support this setting.
}
void loop() {
  int raw=analogRead(A0);
  if(raw<0) { digitalWrite(LED_BUILTIN,LOW); Monitor.println("ADC_ERROR"); return; }
  int request=light_percent(raw, ADC_MAX);
  digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
  Monitor.print("raw="); Monitor.print(raw);
  Monitor.print(", adc_max="); Monitor.print(ADC_MAX);
  Monitor.print(", scaled="); Monitor.println(request);
  // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Five assertions pass; invalid denominator returns -1.

Physical-board check: Build the MCU adapter with the selected board core, then use a 3.3 V-safe LDR divider on A0. Record raw values under two lighting conditions and explain why percentage is not calibrated lux.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 6 — Repair a header and build contract /Slides 110-114

Goal: Two developers duplicate a threshold constant and introduce an incompatible function declaration. Restore one interface and guarded header. K2/A2; suggested practice 20 minutes. Independent folder: labs/activity-06-headers-build.

Exact contract: alarm() accepts signed whole-degree temperature and limit; equality is alarm-on. No hidden global threshold.

Step-by-step Activity 6

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: alarm() accepts signed whole-degree temperature and limit; equality is alarm-on. No hidden global threshold.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.

5. Test your implementation. Run: `sh scripts/test.sh starter`. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read `PROMPTS.md`. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in `docs/review-record.md`. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in `tests/test_logic.c`. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If `adapter/adapter.ino` exists, copy the complete adapter sketch folder to your Arduino sketch workspace with `logic.c` and `logic.h` from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep `.ino` as C++ and `.c` as C.
9. Perform board verification only when equipped. Select UNO Q in the installed Arduino core. Copy both `.c` and `.h` files into the sketch and compile the wrapper. Record the core version and verify C-to-C++ linkage succeeds; do not rename `.c` into `.ino` to conceal an error.
10. Record evidence and handover. Complete `docs/verification-record.md`: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: `alarm()` accepts signed whole-degree temperature and limit; equality is alarm-on. No hidden global threshold. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Repair a header and build contract. Preserve `logic.h`, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: `labs/activity-06-headers-build/Vibe-Coding-Prompts.pdf`. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int alarm(int temperature, int limit) { return temperature >= limit; }
assert(alarm(29,30)==0); assert(alarm(30,30)==1);
assert(alarm(-10,0)==0); assert(alarm(80,30)==1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
```

```

void setup() {
  pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
  Bridge.begin(); Monitor.begin(115200);
  // For ADC activities, verify analogReadResolution(12) is supported,
  // enable it here and keep ADC_MAX consistent before physical use.
}

void loop() {
  int request=alarm(25,30);
  digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
  Monitor.println(request);
  // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Four assertions and duplicate-header inclusion compile successfully.

Physical-board check: Select UNO Q in the installed Arduino core. Copy both .c and .h files into the sketch and compile the wrapper. Record the core version and verify C-to-C++ linkage succeeds; do not rename .c into .ino to conceal an error.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Topic 3 — Controls, memory and data structures /Slides 116-158

K3/A3 /LO3: Select controls, elements and features to meet design objectives (K3/A3).

If-else encodes a boundary policy /Slides 117-119

Upper boundary: At 30.0 degrees the fan request turns on.

Lower boundary: At 28.0 degrees the request turns off.

Deadband: Between thresholds the prior state matters.

Sequence: First-match order must implement the chosen policy.

Contract field	Exact rule
On threshold	≥ 300 tenths.
Off threshold	≤ 280 tenths.
Middle	281..299 retain previous state.
Reset	Initial prior state must be chosen explicitly.

```

if (temp_tenths >= 300) return 1;
if (temp_tenths <= 280) return 0;
return previous != 0;

```

Failure: Threshold chatter. Response: Use two thresholds rather than repeatedly comparing to 300 alone.

Acceptance: 300,290,280 gives on,on,off when initial state is off.

Switch-case selects operating modes /Slides 120-122

Mode: AUTO, MANUAL_ON and OFF have distinct intent.

Default: Unknown values fail off instead of inheriting a stale mode.

Return: Each branch ends without accidental fallthrough.

Validation: A physical toggle encoding must map to these values explicitly.

Contract field	Exact rule
0	Use threshold/hysteresis logic.
1	Manual request on; hardware safety gate still applies.
2	Request off.
Other	Request off and record invalid mode.

```
switch (mode) {  
case 0: return automatic_request;  
case 1: return 1;  
case 2: return 0;  
default: return 0;  
}
```

Failure: Missing break. Response: Use return per branch or explicit breaks; test every branch.

Acceptance: All three valid modes and one unknown mode have separate tests.

For loops walk bounded arrays /Slides 123-125

Initialisation: i starts at first valid element.

Condition: $i < \text{count}$ excludes the one-past-end index.

Progress: Increment occurs once per iteration.

Invariant: count cannot exceed the allocated capacity.

Contract field	Exact rule
Array capacity	4 int elements in the ring buffer.
Valid count	0..4.
Empty	Mean returns 0 before division.
Sum bound	Four values each -1000..1000 yield -4000..4000.

```
int sum=0;  
for (unsigned i=0;i<count;i++) {  
    sum += values[i];  
}  
/* count must be <= capacity */
```

Failure: Off-by-one access. Response: Use $i < \text{count}$, not $i \leq \text{count}$, and test a full array.

Acceptance: Empty, two-element and full four-element buffers produce deterministic means.

While and do-while differ at entry /Slides 126-128

while: May execute zero times when no data exists.

do-while: Always executes once before checking.

Progress: Each iteration must consume or change its termination state.

Embedded: An unbounded retry loop can starve control work.

Contract field	Exact rule
Transport loop	Bound processing by bytes/time per main-loop pass.
No data	while must perform no read.
Retry policy	Maximum attempts and timeout are explicit.
Scheduler	Return control rather than wait indefinitely.

```
while (available > 0) {
    /* consume one bounded item */
    available--;
}
```

```
do { /* one trial */ } while (retry);
```

Failure: Infinite loop. Response: Add a bounded attempt counter and a timeout/fault outcome.

Acceptance: No-data, one-item and budget-exhausted cases all return to the main loop.

Pointers require ownership and length /Slides 129-131

Ownership: The caller allocates and retains the Ring state.

Mutation: ring_push changes the pointed-to object.

Const: ring_mean promises not to modify through its parameter.

Lifetime: The state must outlive each call.

Contract field	Exact rule
Precondition	Pointer non-null and points to a valid initialised Ring.
Aliasing	Other writers must not corrupt count/next.
Array length	Length is not inferred from a bare pointer.
Reset	Initialise values, next and count before use.

```
void ring_push(Ring *r,int value);
int ring_mean(const Ring *r);
/* r points to a valid caller-owned Ring */
/* const blocks writes through this pointer */
```

Failure: Invalid pointer. Response: Validate at the boundary or maintain an explicit caller precondition.

Acceptance: Tests initialise Ring r={{0},0,0} before every independent scenario.

Ring indexing keeps memory fixed /Slides 132-134

Index: next remains 0..3 after modulo.

Overwrite: The fifth sample replaces the oldest slot.

Capacity: No allocation occurs as samples arrive.

Count: Count saturates at four, unlike next which wraps.

Contract field	Exact rule
Initial state	next=0,count=0.
After five pushes	next=1,count=4.
Sequence	10,20,30,40,50 retains 20,30,40,50.
Mean	35 with integer division.

```
r->values[r->next] = value;
```

```
r->next = (r->next + 1u) % 4u;
if (r->count < 4u) r->count++;
```

Failure: Uninitialised index. Response: Initialise the struct; assert next < capacity and count <= capacity.

Acceptance: The fifth push leaves count four and mean35; no array index exceeds3.

Strings reserve a terminator byte /Slides 135-137

Capacity: 16 bytes includes the NUL terminator.

Return: snprintf reports the would-be length excluding NUL.

Truncation: n>=capacity is rejection even if output looks partly correct.

Comparison: strcmp compares text, not pointer addresses.

Contract field	Exact rule
Successful length	5 for T=250.
Required capacity	At least6 for that exact text.
Short capacity	3 must return truncation failure.
Caller	Never use truncated text as an accepted command.

```
char out[16];
int n = snprintf(out,sizeof out,"T=%d",250);
if (n < 0 || (size_t)n >= sizeof out)
    return -1;
/* accepted text: T=250 */
```

Failure: Missing NUL. Response: Use bounded formatting and verify complete length before transmission.

Acceptance: Output equals T=250 and capacity3 returns -1.

Structs package values, not wire bytes /Slides 138-140

Grouping: Fields belong to one sample contract.

Unit: Temperature and humidity use distinct units.

Padding: Compiler may insert bytes between fields.

Transport: Raw sizeof(Sample) is not a portable serial schema.

Contract field	Exact rule
Temperature	-400..1250 tenths.
Humidity	0..100 percent.
Timestamp	uint32_t milliseconds.
CSV	Explicit field order temp_tenths,humidity.

```
typedef struct {
    int temp_tenths;
    int humidity;
    uint32_t stamp;
} Sample;
/* serialize named fields explicitly */
```

Failure: ABI mismatch. Response: Serialise fields with defined units/order rather than copying struct bytes.

Acceptance: Sample {250,60,100} formats exactly 250,60 across host and adapter.

Unions and bitfields need a tag /Slides 141-143

Union: Members share storage; only the active representation is meaningful.

Tag: kind records which member the caller may interpret.

Bitfield: Layout and ordering are implementation-defined.

Portable status: Use masks 1u and 2u for a defined wire byte.

Contract field	Exact rule
Tagged value	Check kind before interpreting value.
Unknown tag	Reject rather than reinterpret bytes.
Bit masks	bit0 fan; bit1 fault; remaining bits zero.
typedef	Names an existing type; does not allocate an object.

```
typedef enum { VALUE_INT, VALUE_FLOAT } Kind;
typedef struct {
    Kind kind;
    union { int whole; float fractional; } value;
} Reading;
/* read the member selected by kind */
```

Failure: Wrong member read. Response: Validate the tag and convert explicitly at an interface.

Acceptance: Status fan1/fault1 is byte3; do not assert a compiler bitfield memory layout.

Activity 7 — Implement fan modes and hysteresis /Slides 144-148

Goal: A server-room fan has AUTO, MANUAL_ON and OFF modes. Temperature noise around the threshold must not chatter the output. K3/A3; suggested practice 30 minutes. Independent folder: labs/activity-07-hysteresis-modes.

Exact contract: Temperature uses tenths of degree; AUTO on at ≥ 300 , off at ≤ 280 , retained state between. Unknown mode fails OFF.

Step-by-step Activity 7

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Temperature uses tenths of degree; AUTO on at ≥ 300 , off at ≤ 280 , retained state between. Unknown mode fails OFF.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.

7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.
9. Perform board verification only when equipped. Use an LED as the fan-request surrogate first. Any real fan requires a correctly rated transistor/MOSFET driver and inductive protection. Inject temperatures 300,290,280 through a test input and capture output history.
10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Temperature uses tenths of degree; AUTO on at ≥ 300 , off at ≤ 280 , retained state between. Unknown mode fails OFF. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement fan modes and hysteresis. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-07-hysteresis-modes/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int fan_next(int mode, int temp_tenths, int previous) {
    switch (mode) {
        case 0: if (temp_tenths>=300) return 1;
                if (temp_tenths<=280) return 0;
                return previous != 0;
        case 1: return 1;
        case 2: return 0;
        default: return 0;
    }
}
assert(fan_next(0,300,0)==1); assert(fan_next(0,290,1)==1);
assert(fan_next(0,280,1)==0); assert(fan_next(0,290,0)==0);
assert(fan_next(1,0,0)==1); assert(fan_next(2,500,1)==0);
assert(fan_next(9,500,1)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
```

```

const int BUTTON_PIN=2;
static uint32_t started=0,last=0;
static int active=0,previous=0;
void setup() {
  pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
  Bridge.begin(); Monitor.begin(115200);
  // For ADC activities, verify analogReadResolution(12) is supported,
  // enable it here and keep ADC_MAX consistent before physical use.
}
void loop() {
  int request=fan_next(0,300,previous);
  digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
  Monitor.println(request);
  // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Seven assertions prove threshold boundaries, retention and unknown-mode OFF.

Physical-board check: Use an LED as the fan-request surrogate first. Any real fan requires a correctly rated transistor/MOSFET driver and inductive protection. Inject temperatures 300,290,280 through a test input and capture output history.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 8 — Use an array-backed ring buffer /Slides 149-153

Goal: Store the latest four environmental samples with fixed memory and compute their integer mean using a pointer to state. K3/A3; suggested practice 30 minutes. Independent folder: labs/activity-08-ringbuffer-pointers.

Exact contract: Buffer capacity is 4; count 0..4; next 0..3; oldest sample overwritten. Empty mean returns 0. Samples are restricted to -1000..1000.

Step-by-step Activity 8

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Buffer capacity is 4; count 0..4; next 0..3; oldest sample overwritten. Empty mean returns 0. Samples are restricted to -1000..1000.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.

6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.

7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Feed a deterministic sequence through Serial or a mock acquisition adapter. Compare logged mean after five samples with 35. Real sensor readings require unit conversion before insertion.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Buffer capacity is 4; count 0..4; next 0..3; oldest sample overwritten. Empty mean returns 0. Samples are restricted to -1000..1000. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Use an array-backed ring buffer. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-08-ringbuffer-pointers/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"

void ring_push(Ring *r, int value) {
    if(value < -1000 || value > 1000) return;
    r->values[r->next] = value; r->next=(r->next+1u)%4u;
    if(r->count<4u) r->count++;
}

int ring_mean(const Ring *r) {
    int sum=0; if(r->count==0) return 0;
    for(unsigned i=0;i<r->count;i++) sum+=r->values[i];
    return sum/(int)r->count;
}

Ring r={{0},0,0}; assert(ring_mean(&r)==0);
ring_push(&r,10); ring_push(&r,20); assert(ring_mean(&r)==15);
ring_push(&r,30); ring_push(&r,40); ring_push(&r,50);
assert(r.count==4); assert(r.next==1); assert(ring_mean(&r)==35);
ring_push(&r,1001); assert(ring_mean(&r)==35);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ adapter around C-compatible logic. Physical result NOT RUN.
#include "logic.h"
#include <Arduino_RouterBridge.h>
Ring samples={{0},0,0};
void setup() {
    Bridge.begin(); Monitor.begin(115200);
    const int values[5]={10,20,30,40,50};
    for(unsigned i=0;i<5;i++) {
        ring_push(&samples,values[i]);
        Monitor.print("mock sample="); Monitor.print(values[i]);
        Monitor.print(", mean="); Monitor.println(ring_mean(&samples));
    }
    Monitor.println("Expected final mean=35; deterministic fixture, not sensor measurement.");
}
void loop() { /* No waiting, allocation or repeated insertion. */ }
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Six assertions pass; fifth sample overwrites 10 and mean is 35.

Physical-board check: Feed a deterministic sequence through Serial or a mock acquisition adapter. Compare logged mean after five samples with 35. Real sensor readings require unit conversion before insertion.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 9 — Encode bounded status records /Slides 154-158

Goal: Create a compact status byte and a human-readable record without serialising compiler-dependent struct memory. K3/A3; suggested practice 25 minutes. Independent folder: labs/activity-09-strings-bitfields.

Exact contract: Temperature text fits caller capacity; snprintf truncation returns -1. Status bit 0=fan, bit 1=fault; other bits zero.

Step-by-step Activity 9

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Temperature text fits caller capacity; snprintf truncation returns -1. Status bit 0=fan, bit 1=fault; other bits zero.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.

6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.

7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Display the exact text through Serial Monitor, with receiver line endings documented. If using an LCD, test display width separately; a 16-byte host buffer is not proof of visible fit.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Temperature text fits caller capacity; snprintf truncation returns -1. Status bit 0=fan, bit 1=fault; other bits zero. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Encode bounded status records. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-09-strings-bitfields/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
#include <stdio.h>
uint8_t status_flags(int fan,int fault) {
    return (uint8_t)((fan?1u:0u) | (fault?2u:0u));
}
int status_text(char *out,size_t cap,int temp) {
    int n=snprintf(out,cap,"T=%d",temp);
    return n<0 || (size_t)n>=cap ? -1 : n;
}
char out[16]; assert(status_flags(1,1)==3);
assert(status_flags(0,1)==2); assert(status_flags(0,0)==0);
assert(status_text(out,sizeof out,250)==5);
assert(strcmp(out,"T=250")==0);
assert(status_text(out,3,250)==-1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ adapter around C-compatible logic. Physical result NOT RUN.
#include "logic.h"
#include <Arduino_RouterBridge.h>
void setup() {
    Bridge.begin(); Monitor.begin(115200);
    char text[16]; int n=status_text(text,sizeof text,250);
    if(n<0) Monitor.println("FORMAT_REJECT");
    else { Monitor.print("mock status="); Monitor.println(text); }
    Monitor.print("flags="); Monitor.println((unsigned)status_flags(1,1));
    // Expected T=250 and flags=3; no raw struct/bitfield serialisation.
}
void loop() { /* Fixture printed once; no LCD rendering claim. */ }
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Six assertions pass; status 3 sets only fan and fault bits; short buffer rejects truncation.

Physical-board check: Display the exact text through Serial Monitor, with receiver line endings documented. If using an LCD, test display width separately; a 16-byte host buffer is not proof of visible fit.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Topic 4 — Contracts, interoperability and fault evidence /Slides 159-201

K4/A4 /LO4: Examine functionality and interoperability of software components (K4/A4).

Serial framing precedes parsing /Slides 160-162

Framing: A parser needs a complete NUL-terminated message.

Capacity: Reserve one byte for NUL and reject overflow.

Line ending: Transport removes newline/CR according to the contract.

Isolation: An invalid frame never partially controls an output.

Contract field	Exact rule
Legal lines	LED=0 and LED=1 only.
Forbidden	Spaces, extra fields and semicolon chaining.
Atomic failure	Previous state remains unchanged.
Budget	Process a bounded number of bytes per loop.

bytes -> fixed-capacity receive buffer

newline -> complete message

overflow -> discard until newline

complete line -> parse_led()

accepted -> bounded LED request

Failure: Split/overlong command. Response: Accumulate complete line safely or discard the entire frame.

Acceptance: Invalid LED=1;PUMP=1 does not change the prior value.

An exact parser rejects extra tokens /Slides 163-165

Equality: Whole-string matching avoids accepting a dangerous prefix.

Allowlist: Only two exact commands are valid.

Return: 1 accepted;0 rejected.

State: No assignment occurs on rejection.

Contract field	Exact rule
Input	Valid NUL-terminated bounded string.
Output pointer	Valid caller-owned int.
Accepted	Sets0 or1.
Rejected	Leaves existing output unchanged.

```
if (strcmp(line,"LED=0")==0) {
    *value=0; return 1;
}
if (strcmp(line,"LED=1")==0) {
    *value=1; return 1;
}
return 0;
```

Failure: Prefix acceptance. Response: Replace starts-with checks with exact schema validation.

Acceptance: LED=2, leading space and chained text reject while prior value7 stays7.

Freshness is part of interoperability /Slides 166-168

Age: Timestamp measures sample freshness, not receipt time alone.

Rollover: Unsigned subtraction supports wrap in the bounded-age context.

Unit range: Impossible converted values are rejected.

Consumer: Display and logger need the same validation contract.

Contract field	Exact rule
Fresh	Age0..5000 ms.
Stale	Age5001 ms and above.
Temperature	-400..1250 tenths.
Humidity	0..100 percent.

```
uint32_t age = now - sample.stamp;
int fresh = age <= 5000u;
int units_ok = sample.temp_tenths >= -400
               && sample.temp_tenths <= 1250;
/* display/log only validated sample */
```

Failure: Stale-but-plausible data. Response: Show a fault/stale state rather than treat old temperature as live.

Acceptance: 5000 ms accepts;5001 rejects;2500 tenths rejects.

I2C contracts include electrical levels /Slides 169-171

Bus: Matching function names do not prove matching physical connections.

Address: Module variants can use different addresses.

Voltage: Pull-ups determine bus high level.

Driver: Installed library must support the selected bus instance.

Contract field	Exact rule
Before connection	Read peripheral voltage/interface manual.
Address evidence	Scan only on correctly reviewed wiring.
Display test	Write short known text and inspect rendering.
Interoperability	Document detection and fix, not just final success.

MCU I2C -> address -> peripheral

A4/A5: pullups to 3.3 V only

Qwiic: I2C4 PD13/PD12

LCD address: check actual module

shared bus: unique addresses and safe levels

Failure: Blank LCD. Response: Check address, bus instance, pull-ups and compatible library in that order.

Acceptance: Record actual LCD address and measured visible text; mock tests cannot prove this.

SD logging needs write and readback /Slides 172-174

Serialisation: Write units and field order in the format spec.

Result: A call returning success is not sufficient without readback.

Resource: Bus/chip-select and library version must match hardware.

Failure: Storage loss must not freeze sensor/control work.

Contract field	Exact rule
Schema	temp_tenths, humidity in the host mock.
Target API	Verified selected-core SD library only.
Integrity	Check rows/values and malformed data.
Evidence	Host CSV formatting is distinct from actual card access.

validated Sample -> explicit CSV

logger adapter -> selected SD library

write result -> close/flush

readback -> compare field values

storage fault -> record safe degraded state

Failure: Logging claim without card. Response: Mark physical SD test NOT RUN and retain mock format evidence.

Acceptance: Actual file readback contains the known sample row; otherwise report the observed fault.

Fault gates must fail deterministically /Slides 175-177

Fault: Disconnected sensor suppresses request even when dry.

Range: Invalid percentage cannot trigger actuation.

Timeout: Freshness is an independent condition.

Limit: Pump maximum runtime requires another safety component.

Contract field	Exact rule
Dry	29 percent requests1 if valid/fresh.
Wet boundary	30 percent requests0.
Stale boundary	2001 ms requests0.
Invalid Boolean	sensor_ok2 requests0.

```

if (sensor_ok != 1) return 0;
if (moisture < 0 || moisture > 100) return 0;
if (age_ms > 2000u) return 0;
return moisture < 30;

```

Failure: Unsafe default. Response: Put all validation gates before the positive dry-soil branch.

Acceptance: Dry valid case passes; each fault independently produces0.

A test oracle must be independent /Slides 178-180

Oracle: Expected results come from specification, not from copying function output.

Boundary: Use neighbouring values around each inequality.

Isolation: Change one fault condition at a time.

Regression: Retain old tests when fixing a defect.

Contract field	Exact rule
Positive test	Proves an allowed path works.
Negative test	Proves a forbidden path is blocked.
Mutation check	Change >2000 to >=2000 and observe boundary test failure.
Coverage limit	Passing examples do not prove all hardware safety.

```

input: moisture29, age2000, ok1
requirement: dry AND fresh AND valid
expected: request1
input: moisture29, age2001, ok1
requirement: stale means safe-off
expected: request0

```

Failure: Tautological test. Response: Write expected constants from the requirement before running implementation.

Acceptance: At2000 ms valid dry input1; at2001 ms0 catches the off-by-one mutation.

Functionality indicators are observable /Slides 181-183

Correctness: The expected output must match the user requirement.

Timing: Host due checks are not physical response-time measurements.

Interoperability: Every producer/consumer agrees on units and framing.

Recovery: A fault must not permanently block the main loop.

Contract field	Exact rule
Host evidence	Assertions, warning gate, deterministic fixtures.
Integration evidence	Known sample passes through display/log mocks.

Contract field	Exact rule
Physical evidence	Board timestamps, visible output and storage readback.
Issue report	Symptom, cause, fix and retest for at least two issues.

functional: correct output for input
 timing: response within contract
 data: unit/range/schema consistent
 resource: bounded memory/work
 fault: safe rejection and recovery
 evidence: repeatable readback

Failure: Works-once claim. Response: Repeat a known case and an injected fault; retain both observed outcomes.

Acceptance: Verification report distinguishes logic, integration and physical evidence categories.

Bridge RPC is a value boundary /Slides 184-186

MCU: provide_safe runs handlers safely with shared main-loop state; no nested RPC call in a callback.

Python: from arduino.app_utils import Bridge; call is synchronous and can raise timeout/connection/value errors.

Environment: APP_SOCKET and arduino-router/App Lab environment are prerequisites.

Safety: UART is routed by Zephyr; notify has no response and must not authorise actuation.

Contract field	Exact rule
Python timeout	Explicit1.0s, not default10s; reject timeout/error.
MCU return	RpcCall.result(ref) is blocking, bool success, result read once.
Handler policy	Validate0/1 and use a TTL lease in deterministic MCU logic.
Physical evidence	Host mock is not proof of actual RPC send/receive/readback.

```
#include <Arduino_RouterBridge.h>
Bridge.begin();
Monitor.begin(115200);
Bridge.provide_safe("set_permit", set_permit);
// Python: Bridge.call("set_permit", value, timeout=1.0)
// MCU Bridge.call returns async RpcCall; result(ref) blocks
```

Failure: Assumed SDK. Response: Check official versioned Bridge API and compile a minimal verified example.

Acceptance: Host allowlist passes first; actual RPC requires separate send/receive/readback record.

Activity 10 — Parse a bounded serial command /Slides 187-191

Goal: Separate serial transport from a parser that accepts only LED=0 or LED=1. A model response must never become arbitrary actuator code. K4/A4; suggested practice 30 minutes. Independent folder: labs/activity-10-serial-contract.

Exact contract: Input is NUL-terminated exact text, line terminator removed by transport. Parser does not accept spaces, extra tokens or unbounded commands.

Step-by-step Activity 10

1. Read the specification. Open `data/spec.json` and `data/cases.csv`. State each input unit, valid range, failure result and expected boundary before editing. Contract: Input is NUL-terminated exact text, line terminator removed by transport. Parser does not accept spaces, extra tokens or unbounded commands.
2. Inspect the shared header. Open `starter/logic.h`. The declaration must match `reference/logic.h` exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: `sh scripts/test.sh reference`. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open `starter/logic.c`. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: `sh scripts/test.sh starter`. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read `PROMPTS.md`. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in `docs/review-record.md`. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in `tests/test_logic.c`. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If `adapter/adapter.ino` exists, copy the complete adapter sketch folder to your Arduino sketch workspace with `logic.c` and `logic.h` from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep `.ino` as C++ and `.c` as C.
9. Perform board verification only when equipped. Use a line receiver with fixed buffer length and overflow discard. Send the five host cases over Serial and confirm invalid lines do not toggle the LED. A serial parser alone is not a tested Bridge RPC transport.
10. Record evidence and handover. Complete `docs/verification-record.md`: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Input is NUL-terminated exact text, line terminator removed by transport. Parser does not accept spaces, extra tokens or unbounded commands. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Parse a bounded serial command. Preserve `logic.h`, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: `labs/activity-10-serial-contract/Vibe-Coding-Prompts.pdf`. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
#include <string.h>
int parse_led(const char *line,int *value) {
    if(strcmp(line,"LED=0")==0) { *value=0; return 1; }
    if(strcmp(line,"LED=1")==0) { *value=1; return 1; }
    return 0;
}
int value=7; assert(parse_led("LED=1",&value)==1 && value==1);
assert(parse_led("LED=0",&value)==1 && value==0);
value=7; assert(parse_led("LED=2",&value)==0 && value==7);
assert(parse_led("LED=1;PUMP=1",&value)==0);
assert(parse_led(" LED=1",&value)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ adapter around C-compatible logic. Physical result NOT RUN.
#include "logic.h"
#include <Arduino_RouterBridge.h>
char line[16]; unsigned length=0;
bool overflowed=false; int led_value=0;
void setup() {
    Bridge.begin(); Monitor.begin(115200);
    pinMode(LED_BUILTIN,OUTPUT); digitalWrite(LED_BUILTIN,LOW);
}
void loop() {
    unsigned budget=16; // Bounded byte processing per loop pass.
    while(budget-- > 0 && Monitor.available()>0) {
        int incoming=Monitor.read(); if(incoming<0) break;
        char c=(char)incoming;
        if(c=='\n') {
            if(!overflowed) {
                if(length>0 && line[length-1]=='\r') length--;
                line[length]='\0';
                if(parse_led(line,&led_value)) {
                    digitalWrite(LED_BUILTIN,led_value?HIGH:LOW);
                    Monitor.print("ACCEPT LED="); Monitor.println(led_value);
                } else Monitor.println("REJECT: prior output unchanged");
            } else Monitor.println("REJECT_OVERFLOW: prior output unchanged");
            length=0; overflowed=false;
        } else if(!overflowed) {
            if(length+1u<sizeof line) line[length++]=c;
            else overflowed=true; // Discard whole overlong frame until newline.
        }
    }
}
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through `Arduino_RouterBridge`; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Five assertions pass; invalid text preserves the previous output.

Physical-board check: Use a line receiver with fixed buffer length and overflow discard. Send the five host cases over Serial and confirm invalid lines do not toggle the LED. A serial parser alone is not a tested Bridge RPC transport.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 11 — Integrate sensor display and logging /Slides 192-196

Goal: Sensor, display and logger modules exchange one explicit sample struct. Detect unit mismatch and stale timestamps before display or logging. K4/A4; suggested practice 35 minutes. Independent folder: labs/activity-11-module-integration.

Exact contract: Sample temperature is tenths of degree, humidity percent 0..100, timestamp ms. Valid temperature -400..1250; stale after 5000 ms.

Step-by-step Activity 11

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Sample temperature is tenths of degree, humidity percent 0..100, timestamp ms. Valid temperature -400..1250; stale after 5000 ms.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.
9. Perform board verification only when equipped. Use mocks first, then connect a 3.3 V-compatible LCD and an approved SD interface only after checking their exact manuals. Capture two real interoperability fixes: e.g. scaling conversion and I2C address/pull-up mismatch. Host format tests do not prove SD write or LCD rendering.
10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Sample temperature is tenths of degree, humidity percent 0..100, timestamp ms. Valid temperature -400..1250; stale after 5000 ms. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Integrate sensor display and logging. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-11-module-integration/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
#include <stdio.h>
int sample_valid(const Sample *s,uint32_t now) {
    return s->temp_tenths>=-400 && s->temp_tenths<=1250 &&
        s->humidity>=0 && s->humidity<=100 &&
        (uint32_t)(now-s->stamp)<=5000u;
}
int sample_csv(char *out,size_t cap,const Sample *s) {
    int n=sprintf(out,cap,"%d,%d",s->temp_tenths,s->humidity);
    return n<0 || (size_t)n>=cap ? -1 : n;
}
Sample s={250,60,100}; char out[32];
assert(sample_valid(&s,5100)==1); assert(sample_valid(&s,5101)==0);
assert(sample_csv(out,sizeof out,&s)==6); assert(strcmp(out,"250,60")==0);
s.humidity=101; assert(sample_valid(&s,100)==0);
s.humidity=60; s.temp_tenths=2500; assert(sample_valid(&s,100)==0);
assert(sample_csv(out,2,&s)==-1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ adapter around C-compatible logic. Physical result NOT RUN.
#include "logic.h"
#include <Arduino_RouterBridge.h>
void present_mock(const Sample *sample,uint32_t now) {
    char csv[32];
    if(!sample_valid(sample,now)) { Monitor.println("SAMPLE_REJECT"); return; }
    if(sample_csv(csv,sizeof csv,sample)<0) { Monitor.println("FORMAT_REJECT"); return; }
    Monitor.print("LCD MOCK:"); Monitor.println(csv);
    Monitor.print("SD_LOG MOCK:"); Monitor.println(csv);
    // These are console mocks, not a physical LCD or SD write.
}
void setup() {
    Bridge.begin(); Monitor.begin(115200);
    Sample valid={250,60,100}; present_mock(&valid,5100);
    present_mock(&valid,5101); // Deliberately stale: reject.
    Sample wrong_units={2500,60,100}; present_mock(&wrong_units,100);
    Sample wrong_humidity={250,101,100}; present_mock(&wrong_humidity,100);
}
void loop() { /* Deterministic integration fixture runs once. */ }
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Seven assertions pass; 5000 ms valid, 5001 ms stale; 2500 and humidity 101 rejected.

Physical-board check: Use mocks first, then connect a 3.3 V-compatible LCD and an approved SD interface only after checking their exact manuals. Capture two real interoperability fixes: e.g. scaling conversion and I2C address/pull-up mismatch. Host format tests do not prove SD write or LCD rendering.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 12 — Build a fault-injection gate /Slides 197-201

Goal: An irrigation request must fail off for stale, disconnected or invalid sensing. Test a guard separately from the pump driver. K4/A4; suggested practice 30 minutes. Independent folder: labs/activity-12-fault-tests.

Exact contract: Moisture 0..100 percent; sensor_ok 0/1. Output is ON only below 30 with age<=2000 ms and sensor_ok true; all other cases OFF.

Step-by-step Activity 12

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Moisture 0..100 percent; sensor_ok 0/1. Output is ON only below 30 with age<=2000 ms and sensor_ok true; all other cases OFF.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.
8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Use an LED surrogate only for the compulsory gate demonstration. Before any pump deployment add maximum run time, manual isolation and an approved driver. Disconnect the sensor and verify a measured OFF request.
10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Moisture 0..100 percent; sensor_ok 0/1. Output is ON only below 30 with age<=2000 ms and sensor_ok true; all other cases OFF. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Build a fault-injection gate. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-12-fault-tests/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int water_request(int moisture,unsigned age_ms,int sensor_ok) {
    if(sensor_ok!=1 || moisture<0 || moisture>100 || age_ms>2000u) return 0;
    return moisture<30;
}
assert(water_request(29,2000,1)==1); assert(water_request(30,2000,1)==0);
assert(water_request(29,2001,1)==0); assert(water_request(29,0,0)==0);
assert(water_request(-1,0,1)==0); assert(water_request(101,0,1)==0);
assert(water_request(29,0,2)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    Bridge.begin(); Monitor.begin(115200);
    // For ADC activities, verify analogReadResolution(12) is supported,
    // enable it here and keep ADC_MAX consistent before physical use.
}
void loop() {
    int request=water_request(29,0,1);
    digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
    Monitor.println(request);
    // Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Seven assertions pass; each injected invalid/stale condition returns OFF.

Physical-board check: Use an LED surrogate only for the compulsory gate demonstration. Before any pump deployment add maximum run time, manual isolation and an approved driver. Disconnect the sensor and verify a measured OFF request.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Topic 5 — User-aligned documentation and bounded extension /Slides 203-231

K5/A5 /LO5: Generate detailed design documentation mapped to user specifications (K5/A5).

Trace requirements into maintenance docs /Slides 204-206

Trace: Each user specification has an implementation and test reference.

Coverage: Untested physical requirements remain explicitly open.

Maintenance: A new developer can locate thresholds, units and interfaces.

AI: Generated prose is reviewed against code, not accepted as authority.

Contract field	Exact rule
Annotated source	Why the guard exists and what units it expects.
Architecture	Data path, ownership and hardware boundary.
I/O specification	Ranges, polarity, timing and driver assumptions.
Project report	Evidence, known limits, changes and support.

R1 dry soil -> safe_request -> test29 -> ON
R2 wet soil -> safe_request -> test30 -> OFF
R3 fault/disable -> guard -> invalid tests -> OFF
R4 hardware timing -> board record -> NOT RUN
R5 maintenance -> header + diagram + report

Failure: Unmapped narrative. Response: Add a requirement ID and evidence link to every claimed behaviour.

Acceptance: R1/R2/R3 map to actual assertions; hardware requirement status remains truthful.

Architecture diagram is not a wiring plan /Slides 207-209

Data flow: Arrows describe values and function boundaries.

Electrical: Voltage/current/wiring is documented separately.

Ownership: One module owns each input conversion and output driver.

Scenario: Illustration helps context but is not an exact board schematic.

Contract field	Exact rule
Sensor input	Raw count and conversion setting.
Logic input	Percentage0..100 plus freshness.
Actuator output	Boolean request to a rated driver.
Physical review	Exact pinout and peripheral manuals are required.

soil sensor adapter -> percentage
percentage -> valid/fresh gate
gate -> request policy
request -> driver adapter
status -> LCD/log adapters
power safety -> separate approved circuit

Failure: Illustration as wiring. Response: Label AI-generated scenario imagery conceptual, not hardware reference.

Acceptance: Maintainer can identify data units and unresolved power/driver checks from the docs.

Local AI proposes; deterministic C decides /Slides 210-212

Beginner scope: Extension reuses simple if logic and clear data contracts.

Local model: Model availability/speed are not assumed or measured here.

Safety: No shell, network, pump or arbitrary code commands.

Fallback: Mock data keeps the activity runnable without a model.

Contract field	Exact rule
Value	Exactly0 or1.
Confidence	Teaching field80..100; not calibrated accuracy.
Age	0..1000 ms.
Rejection	-1; no actuator change from an invalid proposal.

optional local model -> mock/real proposal
proposal {value,confidence,age_ms}
policy: LED only, value0/1
confidence80..100, age<=1000
invalid -> reject (-1)
accepted -> bounded MCU request

Failure: Untrusted natural language. Response: Parse into a fixed schema and validate before any MCU call.

Acceptance: Value2,confidence79/101 and age1001 reject; boundary80/1000 accepts.

User documentation needs honest limits /Slides 213-215

Limit: Do not convert an unrun test into a success statement.

Separation: Compile, upload, function and safety approvals differ.

Support: Include reproduction steps and observed symptom.

Handover: The client can see what remains to be verified.

Contract field	Exact rule
Record fields	Date, variant, revision, tools, fixture and result.
Failure record	Expected, actual, cause, fix and retest.

Contract field	Exact rule
Privacy	No NRIC, credentials or private assessment answers in learner repo.
Release	Course version2.0 and dated change record.

Host logic tests: executed / pass count
MCU compile: recorded / not run
Board upload: recorded / not run
LCD and SD: observed / not run
Pump deployment: safety approval required
Local AI performance: not measured

Failure: Overclaimed completion. **Response:** Replace claims with precise executed evidence and explicit open checks.

Acceptance: Another trainer can reproduce host results and identify physical checks still open.

Document changes preserve the contract /Slides 216-218

Change request: Threshold changes start in the specification.

Regression: Retain fault and timeout tests unchanged.

Traceability: Update code, header, diagram/report and acceptance together.

Release: Slide references come from actual slide_map.json, not guessed numbers.

Contract field	Exact rule
Before change	29/30 test the original boundary.
After approved change	34/35 test the new boundary.
Unchanged safety	Invalid/stale input remains off.
Documentation	Date, reason, author and evidence are recorded.

```
/* proposed change: dry threshold30 ->35 */
/* update requirement R1 */
/* add tests34/35 */
/* review pump safety impacts */
/* regenerate docs and slide references */
/* versioned reviewed release */
```

Failure: Docs-only threshold edit. **Response:** Do not alter documentation independently from code and acceptance tests.

Acceptance: An approved change has consistent requirement, implementation, tests and version record.

Doxygen tags document the public interface /Slides 219-221

Interface: Tags state units and exact return semantics.

Requirement: Add IDs linking user need to implementation and tests.

Generated docs: A documentation tool formats comments; it does not prove truth.

Review: Compare every generated claim with code and executed evidence.

Contract field	Exact rule
Input	Annotated logic.h and logic.c.
Output	Readable function documentation and source links.

Contract field	Exact rule
Validation	Check threshold and failure policies in generated text.
Optional tool	Doxygen installation is not needed for the host C gate.

```
/** @brief Validate a watering request.
 * @param moisture Percentage0..100.
 * @param age_ms Sample age in milliseconds.
 * @return 1 only if dry,valid,fresh;else0.
 * @note No GPIO side effects;not full pump safety.
 */
```

Failure: Generated misinformation. Response: Correct source comments and regenerate; never patch only the rendered report.

Acceptance: Maintenance report includes header,architecture,I/O,trace matrix and truthful unrun checks.

Activity 13 — Document an irrigation interface /Slides 222-226

Goal: A maintenance team needs annotated headers, I/O specifications, a system diagram and a report tracing user requirements to tests. K5/A5; suggested practice 30 minutes. Independent folder: labs/activity-13-documentation-capstone.

Exact contract: Named requirement IDs R1/R2/R3 map to dry/wet/fault evidence. safe_request requires enabled and valid moisture; no hardware claims in documentation.

Step-by-step Activity 13

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Named requirement IDs R1/R2/R3 map to dry/wet/fault evidence. safe_request requires enabled and valid moisture; no hardware claims in documentation.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.

9. Perform board verification only when equipped. Complete docs/verification-record.md with board model, core version, wiring review and measured LED-surrogate behaviour, or write NOT RUN with reason. Do not mark pump safety complete from these host tests.

10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Named requirement IDs R1/R2/R3 map to dry/wet/fault evidence. safe_request requires enabled and valid moisture; no hardware claims in documentation. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Document an irrigation interface. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-13-documentation-capstone/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
/* R1: dry soil requests water; R2: wet soil stays off.
 * R3: disabled/invalid sensing fails off. Unit: percent. */
int safe_request(int enabled,int moisture) {
    if(enabled!=1 || moisture<0 || moisture>100) return 0;
    return moisture<30;
}
assert(safe_request(1,29)==1); assert(safe_request(1,30)==0);
assert(safe_request(0,29)==0); assert(safe_request(1,-1)==0);
assert(safe_request(1,101)==0);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ wrapper around C-compatible logic; LED surrogate only.
// Verify selected UNO Q core/pinout. Do not connect motors or pumps directly.
#include "logic.h"
#include <Arduino_RouterBridge.h>
const int BUTTON_PIN=2;
void setup() {
    pinMode(LED_BUILTIN,OUTPUT); pinMode(BUTTON_PIN,INPUT_PULLUP);
    Bridge.begin(); Monitor.begin(115200);
    // For ADC activities, verify analogReadResolution(12) is supported,
    // enable it here and keep ADC_MAX consistent before physical use.
}
void loop() {
    int request=safe_request(1,29);
```

```
digitalWrite(LED_BUILTIN,request>0?HIGH:LOW);
Monitor.println(request);
// Do not infer buzzer/fan/pump drive approval from this LED surrogate.
}
```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Expected host result: Five assertions pass; documentation contains R1/R2/R3 trace rows and NOT RUN board fields until measured.

Physical-board check: Complete docs/verification-record.md with board model, core version, wiring review and measured LED-surrogate behaviour, or write NOT RUN with reason. Do not mark pump safety complete from these host tests.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Activity 14 — Bound a UNO Q local-AI suggestion /Slides 227-231

Goal: Optional extension: a Linux-side AI model suggests a low-power LED state. C-compatible policy validates value, confidence and age before any MCU request. K5/A5 K4/A4; suggested practice 25 minutes. Independent folder: labs/activity-14-unoq-bounded-ai.

Exact contract: Proposal value is 0/1; confidence integer 0..100, minimum 80; age<=1000 ms. Invalid suggestion returns -1 and must not reach an actuator.

Step-by-step Activity 14

1. Read the specification. Open data/spec.json and data/cases.csv. State each input unit, valid range, failure result and expected boundary before editing. Contract: Proposal value is 0/1; confidence integer 0..100, minimum 80; age<=1000 ms. Invalid suggestion returns -1 and must not reach an actuator.
2. Inspect the shared header. Open starter/logic.h. The declaration must match reference/logic.h exactly; do not change types to silence a warning.
3. Run the supplied reference gate. From this Activity folder run: sh scripts/test.sh reference. It must print the Activity PASS label; this is executed host evidence only.
4. Implement the starter. Open starter/logic.c. Replace each marked TODO return/body with your implementation. Keep the header include and implement every declared function. Use the code examples in the Learner Guide to reason about the contract.
5. Test your implementation. Run: sh scripts/test.sh starter. Investigate the first failed assertion by comparing input, expected result and exact inequality. Do not edit expected values merely to make the test pass.
6. Review with an AI assistant. Read PROMPTS.md. Supply only the mock specification, shared header and your source. Save one proposal, one human acceptance/rejection decision and the test result in docs/review-record.md. No paid API account is required: a trainer can demonstrate review or you can do a manual review.
7. Add a boundary or fault test. Add one independent assertion in tests/test_logic.c. Predict the expected value from the specification first. Re-run both starter and reference to show the test is a contract check rather than a private implementation assumption.

8. Prepare the target wrapper. If adapter/adapter.ino exists, copy the complete adapter sketch folder to your Arduino sketch workspace with logic.c and logic.h from your tested implementation. Select the exact UNO Q board/core. Compile before upload. Keep .ino as C++ and .c as C.
9. Perform board verification only when equipped. Run the mock proposal cases without a model first. If the optional local model is unavailable, retain the mock path. Actual Bridge RPC and board LED call need verified installed library APIs and readback; no model speed or hardware performance claim is supplied.
10. Record evidence and handover. Complete docs/verification-record.md: host compiler and output, sketch compile result, exact variant/core/library versions, wiring review, actual physical result or NOT RUN plus reason. Link each requirement to code and tests; never infer hardware success from PASS.

Vibe-coding prompts /same-source prompt PDF

Design contract: Design C-compatible logic for this exact contract: Proposal value is 0/1; confidence integer 0..100, minimum 80; age<=1000 ms. Invalid suggestion returns -1 and must not reach an actuator. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation: Implement Bound a UNO Q local-AI suggestion. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence: Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Prompt PDF: labs/activity-14-unoq-bounded-ai/Vibe-Coding-Prompts.pdf. Use the same contract in all views; save proposal, human review decision and executed test evidence.

```
sh scripts/test.sh reference
sh scripts/test.sh starter
cc --version
```

Reference C logic and host assertions

```
#include "logic.h"
int proposal_led(int value,int confidence,unsigned age_ms) {
    if((value!=0 && value!=1) || confidence<80 || confidence>100 || age_ms>1000u) return -1;
    return value;
}
assert(proposal_led(1,80,1000)==1); assert(proposal_led(0,100,0)==0);
assert(proposal_led(1,79,0)==-1); assert(proposal_led(2,99,0)==-1);
assert(proposal_led(1,101,0)==-1); assert(proposal_led(1,90,1001)==-1);
```

Complete Arduino C++ adapter /physical extension

```
// Arduino C++ adapter around C-compatible logic. Physical result NOT RUN.
#include "logic.h"
#include <Arduino_RouterBridge.h>
uint32_t received_ms=0;
bool leased=false; int permitted=0,led_state=0;
int review_led(int value,int confidence) {
    int request=proposal_led(value,confidence,0u);
    if(request<0) {
        leased=false; permitted=0; led_state=0;
        digitalWrite(LED_BUILTIN,LOW); return 0;
    }
    permitted=request; received_ms=(uint32_t)millis(); leased=true;
    return 1; // Accepted; Python must read back actual state separately.
}
```

```

int read_led() { return led_state; }
void setup() {
  Bridge.begin(); Monitor.begin(115200);
  pinMode(LED_BUILTIN,OUTPUT); digitalWrite(LED_BUILTIN,LOW);
  Bridge.provide_safe("review_led",review_led);
  Bridge.provide_safe("read_led",read_led);
  const int proposals[6][3]={1,80,1000},{0,100,0},{1,79,0},
                             {2,99,0},{1,101,0},{1,90,1001}};
  for(unsigned i=0;i<6;i++) {
    int result=proposal_led(proposals[i][0],proposals[i][1],(unsigned)proposals[i][2]);
    Monitor.print("mock proposal result="); Monitor.println(result);
  }
  Monitor.println("Expected1,0,-1,-1,-1,-1. No AI inference or RPC round trip claimed.");
}
void loop() {
  uint32_t age=(uint32_t)((uint32_t)millis()-received_ms);
  led_state=leased && age<=1000u ? permitted : 0;
  if(age>1000u) leased=false;
  digitalWrite(LED_BUILTIN,led_state?HIGH:LOW);
  // Only bounded LED leases; no pump, shell, network or model text commands.
}

```

Use the full adapter folder with its logic.c and guarded logic.h. Monitor is routed through Arduino_RouterBridge; this wrapper requires the installed supported core/library. Confirm compile, wiring and observed board output separately. LED-only surrogate does not prove real actuator-driver safety.

Optional UNO Q Bridge client /bounded LED lease

Run python3 bridge_client.py first for a local mock; expected results are 1,0,-1,-1,-1,-1, with no hardware/RPC claim. For the optional physical transport test, compile/upload the supplied adapter and run python3 bridge_client.py --bridge inside the configured UNO Q App Lab/arduino-router APP_SOCKET environment. The MCU validates 0/1 and confidence 80..100, timestamps receipt itself, and expires a 1000ms LED-only lease. Read back immediate and expired state; record actual observations or NOT RUN. Errors cannot authorise retry/notify control. No inference model or paid account is required.

```

"""Mock by default; --bridge requires UNO Q App Lab / APP_SOCKET router environment."""
import argparse,time
def mock(value,confidence,age):
    return value if value in (0,1) and 80<=confidence<=100 and 0<=age<=1000 else -1
def main():
    parser=argparse.ArgumentParser(); parser.add_argument('--bridge',action='store_true')
    args=parser.parse_args()
    if not args.bridge:
        cases=[(1,80,1000),(0,100,0),(1,79,0),(2,99,0),(1,101,0),(1,90,1001)]
        print('Mock policy only; no model inference, RPC or hardware claim.')
        print([mock(*case) for case in cases]); return
    from arduino.app_utils import Bridge
    try:
        ack=Bridge.call('review_led',1,90,timeout=1.0)
        actual=Bridge.call('read_led',timeout=1.0)
        print('Observed acceptance/state:',ack,actual)
        time.sleep(1.1) # MCU independently expires the 1000ms lease.
        expired=Bridge.call('read_led',timeout=1.0)
        print('Observed state after expiry:',expired)
    except (TimeoutError,ConnectionError,ValueError) as error:
        print('Transport failure:',type(error).__name__,str(error))
        print('No retry/notify authorises control. MCU independently expires to safe OFF.')
    if __name__=='__main__': main()

```

Expected host result: Six assertions pass; all untrusted invalid or late proposals return -1.

Physical-board check: Run the mock proposal cases without a model first. If the optional local model is unavailable, retain the mock path. Actual Bridge RPC and board LED call need verified installed library APIs and readback; no model speed or hardware performance claim is supplied.

Troubleshooting: header/link error requires matching declarations and extern C guards; assertion failure requires unit/boundary/state review; missing board is recorded NOT RUN. Submit implemented starter, extra assertion, host output, review record and requirement-linked verification record.

Quick Command Reference

Command	Purpose
cc --version	Identify host C compiler
sh scripts/test.sh reference	Compile and execute supplied reference assertions
sh scripts/test.sh starter	Compile and execute your implementation assertions

Support and Assessment Flow

TRAQOM course prompt -> assessment digital attendance -> WA then PP -> submit according to LMS/assessor-issued instructions -> sign Assessment Summary Record. Course material and assessment: <https://lms-tms.tertiaryinfotech.com/>

Support: enquiry@tertiaryinfotech.com /+65 6100 0613 /www.tertiarycourses.com.sg. Repository: <https://github.com/tertiarycourses/TGS-2023039924-AI-Assisted-C-Programming-for-Arduino>

Sources and Verification Limits

Course identity and learning outcomes: <https://www.tertiarycourses.com.sg/wsqa-assisted-c-programming-for-arduino.html> . Original approved WA/PP instruments and Assessment Plan determine assessment scope; no trainer answers are included. Current June2026 Arduino manual takes precedence over older pinout warnings and marketing OS claims. The core uses C-compatible logic and C++ adapters; requested C++ tutorials provide additional learning context, not different assessment requirements.

Arduino UNO Q hardware documentation: <https://docs.arduino.cc/hardware/uno-q/>

Arduino UNO Q product overview (marketing; current manual wins on OS/electrical facts): <https://www.arduino.cc/product-uno-q>

Qualcomm Dragonwing / UNO Q developer context:

<https://www.qualcomm.com/developer/blog/2025/10/meet-arduino-uno-q-qualcomm-dragonwing-fueled-ai-in-a-blink>

Arduino current UNO Q datasheet / electrical authority:

<https://docs.arduino.cc/resources/datasheets/ABX00162-datasheet.pdf>

Electromaker secondary product context, not electrical authority:

<https://www.electromaker.io/blog/article/arduino-uno-q-specs-features-and-why-qualcomms-ai-partnership-matters>

Arduino5 June2026 local-intelligence context / no benchmark assumed:

<https://blog.arduino.cc/2026/06/05/beyond-edge-ai-bringing-local-intelligence-to-arduino-uno-q/>

Arduino9 June2026 local-AI-agent context / optional experimental extension:

<https://blog.arduino.cc/2026/06/09/local-ai-agents-on-arduino-uno-q/>

Arduino UNO Q store / product variants: <https://store-usa.arduino.cc/pages/uno-q>

Google Books bibliography only: Sean Mitchell, Arduino UNO Q Projects for Beginners,3

June2026,ISBN9798199919678; no full text or projects copied:

[https://books.google.com/books/about/Arduino_UNO_Q_Projects_for_Beginners.html?](https://books.google.com/books/about/Arduino_UNO_Q_Projects_for_Beginners.html?id=lgYX0gEACAAJ)

[id=lgYX0gEACAAJ](https://books.google.com/books/about/Arduino_UNO_Q_Projects_for_Beginners.html?id=lgYX0gEACAAJ)

Google C++ educational materials / primary learning reference: <https://developers.google.com/edu/c++>

Learn C++ requested supplementary tutorial: <https://www.learn-cpp.org/>

Google C++ Getting Started / primary compilation context:

<https://developers.google.com/edu/c++/getting-started>

LearnCpp requested supplementary tutorial: <https://www.learncpp.com/>

Arduino RouterBridge official source,commitb8ec5b4b92f478981278c82bda0b3338505d16ba:

https://github.com/arduino-libraries/Arduino_RouterBridge/tree/b8ec5b4b92f478981278c82bda0b3338505d16ba

Arduino Python Bridge official source,commit6a3c2088ef8c319d8f74620716f0f6c187f36cba:

<https://github.com/arduino/app-bricks-py/tree/6a3c2088ef8c319d8f74620716f0f6c187f36cba>

ArduinoCore-zephyr official UNO Q pin/PWM

configuration,commit79b3f1afd455f55e4a25030953617152c0227c:

https://github.com/arduino/ArduinoCore-zephyr/blob/79b3f1afd455f55e4a25030953617152c0227c/variants/arduino_uno_q_stm32u585xx/arduino_uno_q_stm32u585xx.overlay

AI-generated conceptual workbench and irrigation imagery are illustrative, not exact board/pinout/wiring or measured performance evidence. All physical board, model speed, LCD and SD claims require separate observed tests. Practice exam is omitted unless a matching official exam is directly verified.